

Extensibility and Platformization

Review the codebase from the perspective of building a reusable AI platform.

Explain:

- Which parts are extensible
- Where plugins/adapters could fit
- Whether model providers can be swapped cleanly
- Whether storage/retrieval backends can be swapped
- Whether workflows can be composed
- Whether new AI services can be added without major rewrites

Then evaluate:

- Is this really a platform architecture or still an application architecture?
- What refactors would make it truly platform-grade?

Extensibility and Platformization Review

Which Parts Are Extensible

LLM providers — genuinely extensible.

LLMService.getProvider(model) routes by model name prefix. Adding a new provider is:

1. Implement generateResponse() and generateStreamResponse() on a new class
2. Add a branch in getProvider()
3. Add the client field and instantiation in the constructor

No other files change. The UserApiKeys interface would need a new key field, but the propagation is clean. This is the best-designed extension point in the codebase.

RAG modes — extensible by config.

rag-modes.ts is a pure data file: a Record<RAGModelId, RAGModeConfig>. Adding a 5th mode is adding one object to the record and updating the union type. No route code changes. This is close to a plugin pattern, just without a registration mechanism.

Storage backends — extensible by interface.

StorageProvider is a clean interface with 4 methods (uploadFile, deleteFile, getFileInfo, downloadFile). createStorageProvider() selects the implementation via config. Adding a new provider (e.g., GCS) means implementing the interface and adding a case in the factory. Clean.

Workflow node types — extensible by switch.

executeNode() in workflow-executor.ts is a switch on node.type. New node types are added by adding a case. The node inputs/outputs flow through a shared context object. It works but it's not truly plugin-like — you can't register a node type from outside the file.

Where Plugins/Adapters Could Fit

LLM providers already follow the adapter pattern and could accept external registration:

// What could exist:

```
llmService.registerProvider('my-provider', (model, apiKey) => new MyProvider(apiKey));
```

RAG retrievers are currently hardcoded as enhanced-rag-service.ts. A retriever interface (retrieve(query, options): Chunk[]) would let you swap in BM25-only, graph-based retrieval, or external vector stores (Pinecone, Weaviate) without touching route code.

Workflow node types could use a registry pattern:

// What could exist:

```
workflowExecutor.registerNodeType('http-request', async (node, ctx) => { ... });
```

Document parsers in document-processor.ts are a switch on MIME type. A parser registry (Map<mimetype, Parser>) would let new file types be added without touching the core processor.

Embedding models in embedding-service.ts hardcode Cohere + OpenAI. An EmbeddingProvider interface (same shape as LLMProvider) would let the embedding backend swap alongside or independently of the LLM backend.

Whether Model Providers Can Be Swapped Cleanly

Yes, for generation. No, for embedding.

Generation: swapping providers only requires the model name prefix to route to the new class. If a user brings their own Mistral API endpoint, you add one class and one branch. Clean.

Embedding: EmbeddingService hardcodes cohere.embed() with Cohere-specific API shape and openai.embeddings.create() with OpenAI-specific shape. There is no EmbeddingProvider interface. If you want to use a different embedding model (e.g., text-embedding-3-large, a local model, or Voyage AI), you edit embedding-service.ts directly. This is not swappable — it's hardwired.

Additionally: the vector dimension is baked into the schema (vector(1024) for Cohere embed-english-v3.0). Switching to OpenAI text-embedding-3-small (1536d) or text-embedding-3-large (3072d) requires a DB migration and full re-indexing of all document chunks.

Whether Storage/Retrieval Backends Can Be Swapped

Storage: yes. The StorageProvider interface is clean. Swapping R2 for S3 or GCS is one factory case.

Retrieval: partially. EnhancedRAGService is tightly coupled to:

- PostgreSQL via Drizzle ORM
- pgvector's <=> cosine operator syntax
- The document_chunks and files table shapes

Swapping to Pinecone, Weaviate, or Qdrant would require rewriting EnhancedRAGService completely. There is no VectorStore interface that the retriever implements. If you want Pinecone in production and pgvector in development, there is no clean swap path — you'd be running two codepaths with no abstraction layer between them.

Whether Workflows Can Be Composed

Currently: linear composition only.

The DAG executor supports parallel branches and conditional routing. Workflows can chain nodes. However:

- Workflows cannot call other workflows (no workflow node type)
- There is no workflow library or template system beyond the isTemplate flag
- Workflow I/O is an untyped Record<string, any> — there is no schema validation on node inputs/outputs
- There is no retry/error-recovery node type — a failed node fails the entire workflow
- Notebooks and workflows are completely separate systems with no integration path

True workflow composition — where a workflow is itself a node, or where a notebook cell can trigger a workflow — is not possible without structural additions.

Whether New AI Services Can Be Added Without Major Rewrites

Depends on the service type:

Is This a Platform Architecture or an Application Architecture?

It is an application architecture with platform ambitions.

Platform architectures have three characteristics that are absent here:

1. Extension without modification.

A true platform lets external code extend it without touching core files. Here, adding a new LLM provider means editing llm-providers.ts. Adding a new RAG mode means editing rag-modes.ts. Adding a new workflow node means editing workflow-executor.ts.

All extension points require modifying core files. There is no plugin registry, no extension API, no hook system.

2. Composable primitives.

A platform exposes building blocks that can be combined arbitrarily. The current primitives (RAG, chat, workflows, notebooks) are independent features, not composable units. A workflow cannot embed a RAG query. A notebook cannot trigger a workflow. A

RAG search cannot be composed with a custom retriever. Each feature is a vertical silo.

3. Tenant-scoped configuration.

An enterprise platform allows each tenant (organization) to configure their own LLM providers, retrieval settings, storage backends, and rate limits. Currently all of this is global config. One GROQ_API_KEY applies to all users. One STORAGE_PROVIDER applies to everything. The tier system enforces limits but not per-tenant configuration surfaces.

The shared/sdk/, domain-events.ts, the organizations/workspaces tables in the schema, and the WorkspaceMode type in access-control.ts all signal that a platform architecture is intended. The schema has organizations, workspaces, memberships, and inference_runs tables. But none of these are wired up in the application layer — routes still use userId as the root identity and ignore organizationId entirely.

Refactors That Would Make It Truly Platform-Grade

1. Publish the shared SDK.

shared/sdk/src/events/ and domain-events.ts already define the event contract. Wire up emitDomainEvent() in the routes. Register consumers in shared/sdk. This gives the platform an event backbone that external integrations and internal services can hook into without touching route code.

2. Create an EmbeddingProvider interface parallel to LLM providers.

```
interface EmbeddingProvider {  
  embed(texts: string[]): Promise<number[][]>;  
  dimensions: number;  
}
```

Register providers the same way LLM providers are registered. This makes embedding model swaps config-driven, and decouples the DB schema dimension from the service code.

3. Create a VectorStore interface.

```
interface VectorStore {  
  upsert(chunks: Chunk[]): Promise<void>;  
  search(query: number[], options: SearchOptions): Promise<ScoredChunk[]>;  
}
```

EnhancedRAGService becomes an orchestrator that calls vectorStore.search(). Swapping Postgres for Pinecone is a config change.

4. Wire the organizations/workspaces schema to the application.

The tables exist. Adding organizationId to the query root identity and enforcing it in requireTier() and ownership checks is the path to real multi-tenancy. Without this, the enterprise tier is billing tier, not isolation tier.

5. Add a workflow node registry.

```
workflowExecutor.register('rag-search', ragSearchNodeHandler);  
workflowExecutor.register('llm-generate', llmNodeHandler);  
workflowExecutor.register('http-request', httpNodeHandler);
```

This makes workflow node types addable from outside the executor, and makes them independently testable.

6. Add sub-workflow and notebook-trigger node types.

Composability: a workflow node that invokes another workflow by ID. A workflow node that runs a notebook cell. This is the difference between a workflow builder and a workflow platform.

Inspect the codebase for external integrations and abstraction quality.

Review:

- AI model providers
- vector databases
- storage providers
- auth providers
- publishing/integration adapters
- message queues
- observability hooks

For each, explain:

- Whether it is abstracted properly
- Whether provider lock-in exists

- Whether the code is ready for multi-provider enterprise use

1. AI Model Providers (LLM Generation)

Files: `services/llm-providers.ts`

Abstraction quality: Good — the best-abstracted integration in the codebase.

Each provider (`OpenAIProvider`, `AnthropicProvider`, `GroqProvider`, `OllamaProvider`, `MockProvider`) implements the same two-method interface (`generateResponse`, `generateStreamResponse`) without being forced into a formal TypeScript interface declaration. The

routing logic in `LLMService.getProvider(model)` is a single responsibility function. Adding a new provider requires editing exactly one file.

Lock-in assessment:

- No SDK is imported outside `llm-providers.ts`. Call sites use `LLMService` only, never SDK classes directly.
- SDK clients are cached per API-key fingerprint (recently fixed).
- The `MockProvider` enables full offline development without any live keys.

What's missing for multi-provider enterprise use:

There is no formal `LLMProvider` interface type that providers are declared to implement. TypeScript doesn't enforce the contract — a provider that's missing `generateStreamResponse` would only fail at runtime. A shared interface would catch this at compile

time and make adding providers self-documenting.

Provider selection is hardcoded by model name prefix, not by user configuration. An enterprise tenant that wants to route all Anthropic calls through their own AWS Bedrock endpoint, or wants GPT-4 calls to go through Azure OpenAI, has no mechanism for this. There is no per-tenant LLM routing layer.

The `ollamaBaseUrl` is global config — only one Ollama endpoint for the entire platform. Enterprise customers with private hosted models need per-org endpoint configuration.

Verdict: Ready for current multi-provider use. Not ready for per-tenant routing or custom endpoints.

2. Embedding Providers

Files: `services/embedding-service.ts`

Abstraction quality: Poor — hardwired, not abstracted.

EmbeddingService has no interface. It hardcodes Cohere's embed-english-v3.0 as primary and OpenAI's text-embedding-ada-002 as fallback. The fallback chain is implicit: try Cohere → try OpenAI → generate mock embeddings (deterministic sin-based hash).

This fallback is silent — if Cohere fails in production and OpenAI takes over, the embedding dimensions change (1024 Cohere vs 1536 OpenAI/Ada-002), which is structurally incompatible with the pgvector schema (vector(1024)). The fallback to OpenAI would silently produce wrong-dimension vectors, causing all similarity searches to return garbage or error.

Lock-in assessment:

- Hard lock-in to Cohere as primary. Swapping to Voyage AI, Jina, or a local embedding model requires editing embedding-service.ts directly.
- The vector(1024) dimension is baked into the database schema in migration 0003. Changing providers means a DB migration and full re-indexing of every document chunk already stored.
- EmbeddingService creates new EmbeddingService(userApiKeys) per call (unlike LLMService which now has a cache). Every embedding call for user documents spins up new Cohere and OpenAI SDK clients.

Verdict: Single-provider lock-in with a broken fallback. Not ready for multi-provider or enterprise use. The schema-level vector dimension is the most expensive lock-in to undo.

3. Vector Database / Retrieval Backend

Files: services/enhanced-rag-service.ts, services/rag-service.ts

Abstraction quality: None — directly coupled to PostgreSQL + pgvector.

There is no VectorStore interface. EnhancedRAGService uses Drizzle ORM directly, constructs pgvector-specific SQL operators ($\Leftrightarrow$ cosine distance), and references the document_chunks and files table schemas directly. The FTS component uses

PostgreSQL-specific to_tsvector and to_tsquery syntax. Two retrieval implementations exist (rag-service.ts legacy, enhanced-rag-service.ts active) with no shared abstraction between them.

Lock-in assessment:

- Complete lock-in to PostgreSQL. Moving to Pinecone, Weaviate, Qdrant, Milvus, or ChromaDB requires a complete rewrite of retrieval.
- The hybrid scoring formula ($\text{vectorWeight} * \text{cosine_sim} + \text{keywordWeight} * \text{ts_rank}$) is expressed as raw SQL that only makes sense in Postgres.
- There is no way to A/B test retrieval backends or run a different vector store per enterprise tenant.

Verdict: Deeply coupled. Not a platform-grade integration. For enterprise customers who require an external vector store (common for data-sovereignty reasons), there is no path.

4. Storage Providers

Files: `services/file-storage.ts`, `services/local-file-storage.ts`

Abstraction quality: Good — the cleanest integration boundary in the codebase.

`StorageProvider` is a formal TypeScript interface with 4 methods. Three concrete implementations exist (`CloudflareR2Storage`, `S3Storage`, `LocalFileStorage`). The factory `createStorageProvider()` selects by config. Nothing outside `file-storage.ts` imports a specific provider class — all call sites use the `storageProvider` singleton typed as `StorageProvider`.

A second factory (`createTrainingStorageProvider`) correctly separates training data storage from document storage using a separate bucket, with a graceful fallback.

Lock-in assessment:

- No lock-in. Swapping the provider is a config change (`STORAGE_PROVIDER=aws-s3`).
- Adding GCS, Azure Blob, or a custom S3-compatible endpoint is one new class + one factory case.
- `getPresignedUrl` is on the interface as optional (?) — `LocalFileStorage` doesn't implement it, which means attempting to get a presigned URL for a volume-stored file would silently return undefined. This is a gap, not lock-in.

What's missing for enterprise: Per-tenant storage configuration. All users' files go to one bucket. An enterprise customer who needs their data in a dedicated bucket, or a specific AWS region, has no mechanism for this.

Verdict: Well-abstracted. Provider-swappable. Not ready for per-tenant storage isolation.

5. Auth Providers

Files: `services/googleAuthService.ts`, `middleware/auth.ts`

Abstraction quality: Weak — Google is the only external OAuth provider and it is hardwired.

`GoogleAuthService` is a concrete class with no interface. It uses `google-auth-library` directly. The auth flow (`/api/auth/google`, `/api/auth/google/callback`) routes import and instantiate `googleAuthService` specifically. There is no `OAuthProvider` interface that a GitHub, Microsoft, or Okta provider could implement.

Email/password auth lives directly in `auth.ts` as route handlers with `bcrypt`. It is not wrapped behind an interface either.

Lock-in assessment:

- Complete lock-in to Google OAuth as the only social login. Adding GitHub OAuth means adding a `githubAuthService.ts`, new routes, and modifying the user creation flow — with no shared abstraction to guide the shape.

- The server requires `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` at startup (they are required fields with no `.optional()`) — the server won't start without Google credentials even if you only want email/password auth. This is a hard dependency on a specific external provider.

- SAML/SSO for enterprise (SSOProvider integration) is listed as an Enterprise feature on the Pricing page but has zero implementation or interface definition in the codebase.

What's missing for enterprise: SSO/SAML is the table-stakes requirement for enterprise authentication. The current architecture has no path to implementing it short of major additions to the auth system.

Verdict: Hardwired to Google. Google is a startup requirement at startup time regardless of config. Enterprise SSO is aspirational with no implementation path.

6. Message Queue

Files: `services/queue.ts`

Abstraction quality: None — BullMQ is used directly throughout.

BullMQ types (`Queue`, `Worker`, `Job`) are used directly at every call site. There is no `JobQueue` interface. The queue is constructed, jobs are enqueued, and workers are defined all in one file, but that file is directly imported by `index.ts` and `routes/files.ts`.

Lock-in assessment:

- Complete lock-in to BullMQ + Redis. Moving to SQS, RabbitMQ, or Google Cloud Tasks would require rewriting `queue.ts` entirely and updating all import sites.

- The job payload schema (`ProcessDocumentJob`, `IndexDocumentJob`) is typed but not versioned. If the schema changes, in-flight jobs from the old schema would be processed by new workers and likely fail silently.

- Two Redis libraries (`ioredis` for BullMQ, `redis v4` for caching) create redundant connections to the same instance. There is no connection sharing or abstraction that would make it easy to route both through a single connection pool.

Verdict: Deeply coupled to BullMQ/Redis. No path to alternative queue backends. Job schema is unversioned.

7. Observability Hooks

Files: `utils/prometheus-metrics.ts`, `monitoring/metrics.ts`, `middleware/monitoring.ts`, `utils/error-tracking.ts`, `utils/audit.ts`

Abstraction quality: Three-tier system where two tiers are fully functional and one is completely hollow.

Tier 1 — Prometheus metrics (functional but unwired):

`prometheus-metrics.ts` defines 20+ metric instruments: histograms for LLM API duration, vector search duration, document processing time; counters for LLM API calls, RAG documents, HTTP requests. The instruments are exported from the module.

Critical finding: None of these instruments are imported or `.observe()/.inc()` called anywhere in routes or services. The only import of `prometheus-metrics.ts` in the application is `metrics.ts` (the route) which imports `register` to expose the `/metrics`

endpoint. The endpoint returns `collectDefaultMetrics()` output (Node.js process stats) only. All the custom AI-specific metrics — LLM call counts, vector search latency, document processing time — are defined but never recorded. The Grafana dashboards in `monitoring/grafana/` are querying metrics that are never emitted.

Tier 2 — In-memory metrics collector (functional, short-lived):

MetricsCollector in monitoring/metrics.ts collects request/error/usage metrics in memory arrays, capped at 10,000 entries. Used by the monitoring middleware via Fastify hooks. This works but is process-local, not persisted, and resets on every deploy.

It powers the `/api/health/health` endpoint's metrics summary.

Tier 3 — Error tracking (stub):

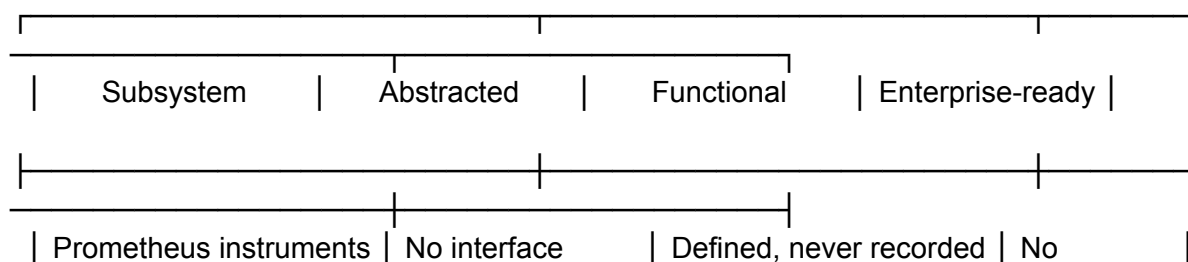
ErrorTrackingService in `utils/error-tracking.ts` is a complete stub. Every method body is a comment showing where Sentry calls would go. In production it logs to Pino instead of sending to any error tracking service. `captureException`, `captureMessage`, `setUserContext`, `addBreadcrumb` all end at `logger.*`. No Sentry DSN, no GlitchTip, no Datadog APM — not even an optional import.

Audit log (functional, well-designed):

audit.ts is the exception — it works correctly. AES-encrypted writes to audit_logs table with typed AuditAction enum, IP address, user agent, resource identifier. Non-fatal (errors are swallowed). The action enum covers auth events and file/notebook

events but is missing RAG operations, workflow executions, and API key changes — events that are high-value for enterprise compliance.

Verdict:



In-memory collector	No interface	Works, process-local	No	
Error tracking	Clean interface shape	Stub only	No	
Audit log	No interface	Works	Partially	

8. Publishing / Integration Adapters

What exists: Nothing. There are no webhook delivery mechanisms, no outbound event publishing, no integration adapters for Slack, Teams, Zapier, or any third-party workflow platform. The domain-events.ts infrastructure (emitDomainEvent, registerDomainEventDispatcher) provides the right internal plumbing — but emitDomainEvent is never called from any route or service. Zero dispatchers are registered.

The shared/sdk/src/events/ directory exists but is empty of implementation. The shared/events/ directory exists but is empty.

This is a significant gap for a platform product. Enterprise customers expect webhooks for document indexing completion, RAG search events, workflow completions, and audit events. There is no delivery infrastructure for any of this.

Summary Table

Integration ready	Abstracted	Lock-in	Multi-provider	
LLM generation providers work today)	Partially (no formal interface)	Low	Yes (4	

Embedding	None	High (Cohere + schema)	No	
Vector store	None	Complete (Postgres/pgvector)	No	
File storage	Well (formal interface)	None	Yes	
Auth (OAuth)	None	Google hardwired	No	
Auth (SSO/SAML)	None	N/A (not implemented)	No	
Message queue	None	Complete (BullMQ/Redis)	No	
Prometheus metrics exist, unwired	None	Low (prom-client)	Instruments	
Error tracking wired)	Interface shape exists	None (stub)	No (nothing	

Audit log	None	Postgres	Partially	
Webhooks/events	Infrastructure exists	N/A (not wired)	No	

The single largest abstraction gap is that the three components most critical to an AI platform — embedding provider, vector store, and event publishing — have zero abstraction. They are the hardest to swap, the most likely to need swapping (for enterprise data-sovereignty requirements), and the most intertwined with the database schema (embedding dimensions). These three should be the first targets for interface extraction before any enterprise contract is signed.